CS 240 Programming Assignment Phase 1

Winter Semester 2025-2026

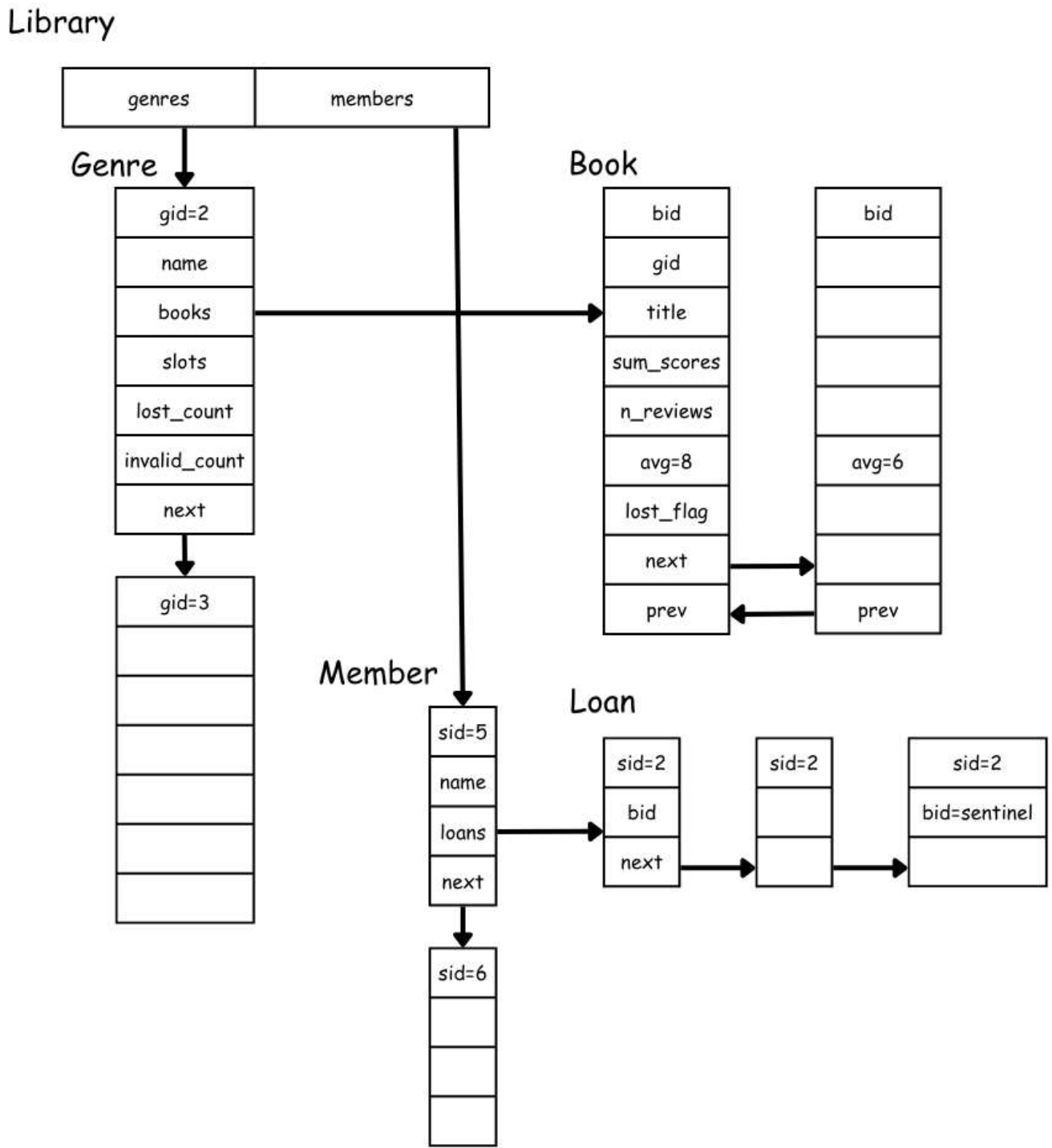
Presenter: Myron Tsatsarakis – myrontsa@csd.uoc.gr

7 October 2025

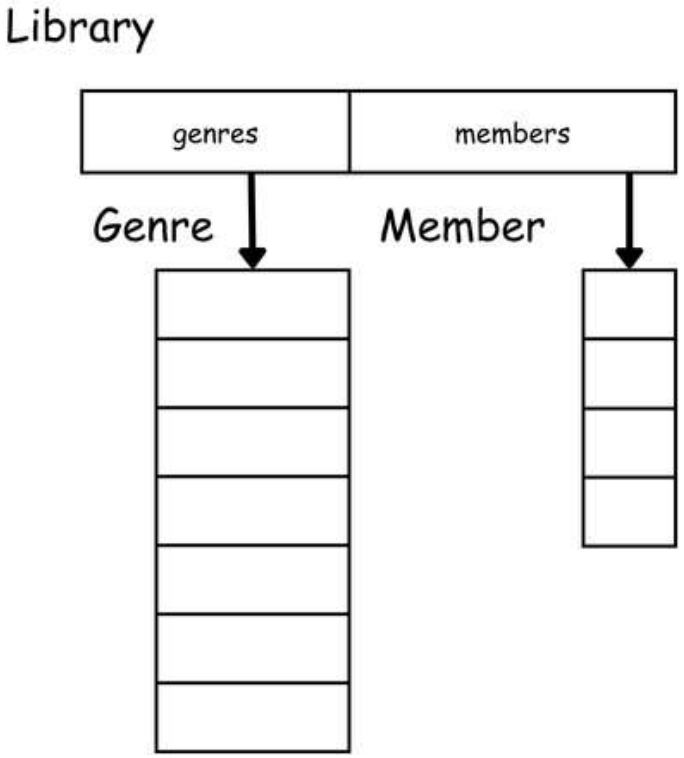
Concept

- **Main Idea:** Library Management System
- Supports
  - Loaning, returning, reviewing books by members
  - Showcasing popular book titles at the building window
- Library: contains Genres list and Members list
- Sorted Genres list
  - Each Genre contains a sorted doubly-linked Books list
- Sorted Members list
  - Each Member contains an unsorted, sentinel-based, Loans list

Design & Data Structures

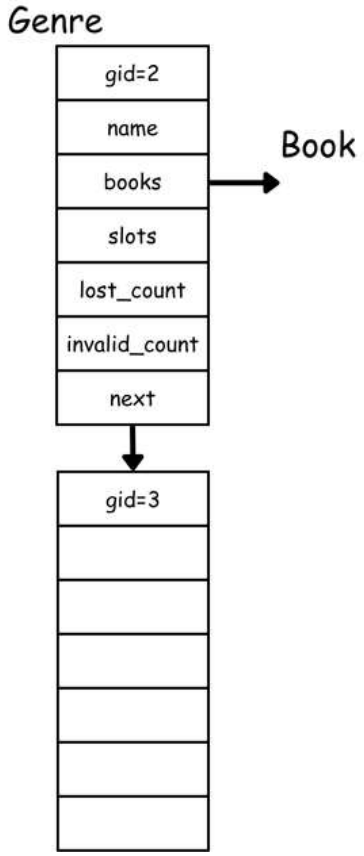

Library
genres
members
Genre
gid=2
name
books
slots
lost_count
invalid_count
next
gid=3
Book
bid
gid
title
sum_scores
n_reviews
avg=8
lost_flag
next
prev
bid
avg=6
prev
Member
sid=5
name
loans
next
sid=6
Loan
sid=2
bid
next
sid=2
sid=2
bid=sentinel

Library



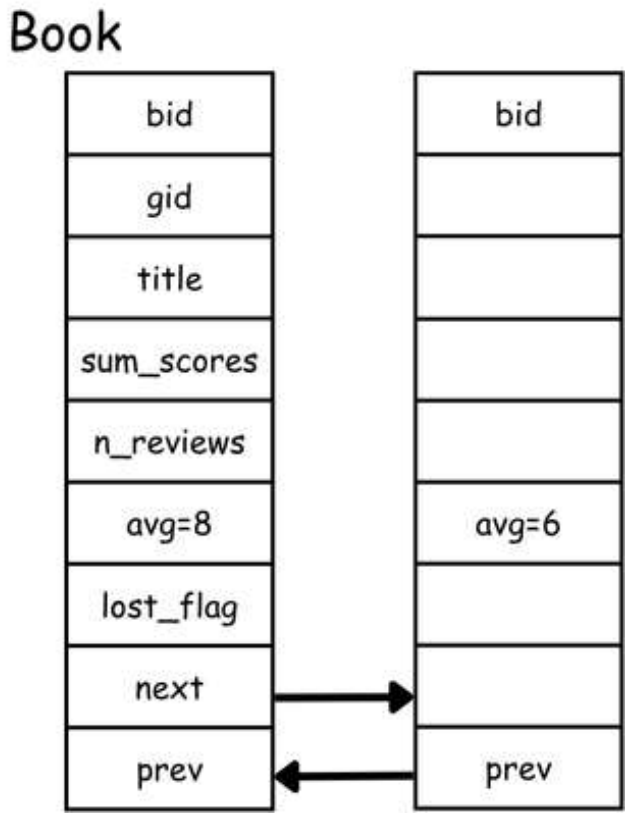
- Type Library
  - struct of 2 pointers to lists
  - **genres** pointer to Genre List
  - **members** pointer to Member List
  - Both lists
    - Singly-Linked
    - Sorted by id (gid or sid)
- 1 global Library instance

Genre List



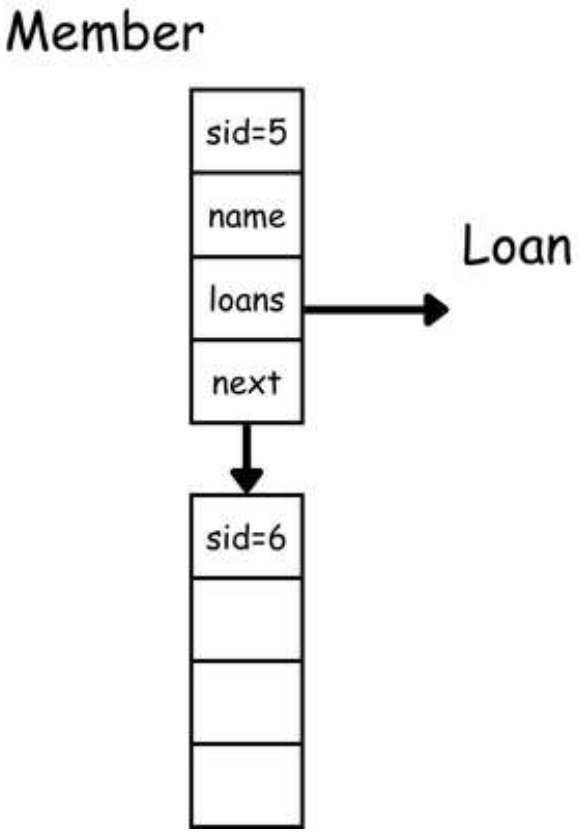
- Genre List
  - Singly-Linked
  - Sorted by **gid**
- Type Genre
  - **gid** unique id
  - **name** string (chars array)
  - **books** pointer to Book List
  - **slots** count of display slots
  - **lost_count** count of lost books
  - **invalid_count** count of invalid reviews
  - **next** pointer to Genre

Book List



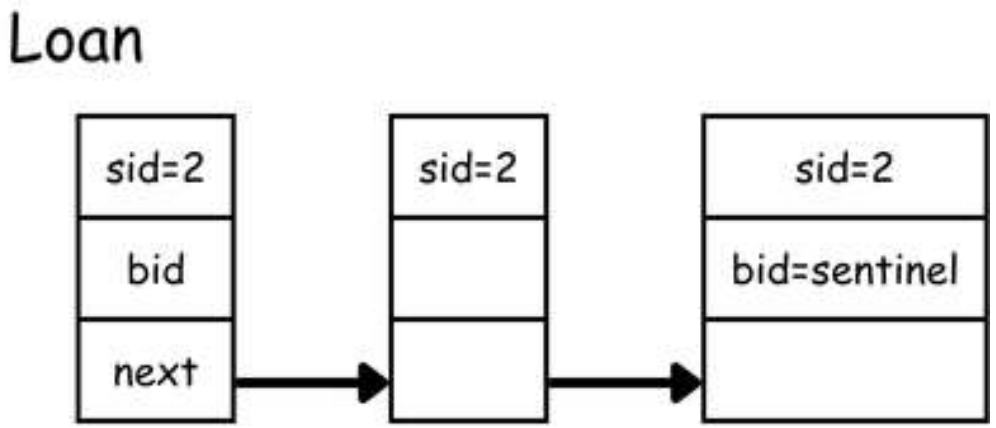
- Book List
  - Doubly-Linked
  - Sorted descending by **avg**
- Type Book
  - **bid** unique id
  - **gid** genre id
  - **title** string (chars array)
  - **sum_scores** sum of received user scores
  - **n_reviews** sum of received user reviews
  - **avg** floor(sum_scores / sum_reviews)
  - **lost_flag** boolean. true if book is reported as lost
  - **next** pointer to Book
  - **prev** pointer to Book

Member List



- Member List
  - Singly-Linked
  - Sorted by **sid**
- Type Member
  - **sid** unique id
  - **name** string (chars array)
  - **next** pointer to Member

Loan List



- Loan List
  - Singly-Linked
  - Sentinel
- Type Loan
  - **sid** member id
  - **bid** book id
  - **next** pointer to Loan

Events

G <gid> <name>
Insert Genre

- Inserts genre **gid**
  - Find genre **gid** in library genres list
  - If it exists, return
  - Insert genre **gid**
  - Sorted Insert

BK <bid> <gid> <title>
Insert Book

- Inserts book **bid** to genre **gid**
  - Find genre **gid** in library genres list
  - Search for book **bid**
  - If it exists, return
  - Initialize book **bid** data to zero
  - Insert book **bid**
  - Sorted Insert

M <sid> <name>
Insert Member

- Inserts member **sid**
  - Find member **sid** in library members list
  - If it exists, return
  - Insert member **sid**
  - Sorted Insert

L <sid> <bid>
Loan Book

- Inserts loan record **sid** of book **bid**
  - Find member **sid** in library members list
  - If not exists, return
  - Find book **bid**, across genres
  - If not exists, return
  - Find loan record of book **bid**, among loan records of member **sid**
  - If it exists, return
  - Insert loan record of book **bid**, to loan records of member **sid**

R <sid> <bid> <score> <status>
Return Book

- Update book and genre stats, and delete loan record
  - Find member **sid** in library members list. If not exists, return.
  - Find loan record of book **bid**, among **sid** loans. If not exists, return.
  - Find book **bid**, across genres. If not exists, return.
  - Store the genre gid containing the book
  - If **status** is lost
    - Set book lost_flag to true. Update gid lost_count. Delete loan record.
  - If status is ok and score is NA. Delete loan record.
  - If status is ok and score is outside [0, 10]. Update gid invalid_count . Delete loan record.
  - If status is ok and score is [0, 10]
    - Update n_reviews = n_reviews + 1
    - Update sum_scores = sum_scores + score
    - Update avg = floor(sum_scores / n_reviews)
    - Reposition book in the gid book list. Book list must be sorted descending by avg.
    - Delete loan record

D
Display Books

- Using the S event, set the total number of display slots as SLOTS
- For each genre g:
  - points = sum(book.sum_scores) for each book where n_reviews > 0 and lost_flag=false
  - quota = floor(points / SLOTS)
  - seats = floor(points / quota).
  - seats are decremented from SLOTS
  - rem = points – (seats * quota)
- Distribute remaining SLOTS to genres with the highest rem values:
  - From highest to lowest rem genres:
    - rem are decremented from SLOTS
    - seats = seats + rem
- The final number of seats for each genre, is stored in the slots field: slots = seats
- For each genre g:
  - Store the top n books with the highest avg score. Where n = g.slots
  - These books can then be printed using the PD event

Coding Tips

- **Understand** your algorithm **before** coding it
- Test your algorithm with simple examples before coding
  - Use pen & paper, draw shapes
- After coding your algorithm test it with various inputs
  - Do this **before** moving the next algorithm
  - Make sure you are confident your code works before continuing
  - Use **gdb** to debug errors
- If an algorithm is too complex, split it to simpler parts
  - Apply the above for each part recursively
- Provided tests are **not enough**
  - Implement **your own**, more complex, tests to verify correctness
  - Try to cover all possible input cases
- Utilize **mailing list** and **office hours**

Examination Tips

- Practice
  - Explaining your algorithms using high-level non-technical language
  - Try it on friends and non-technical people
    - Rubber duck method
  - Make sure your descriptions are understandable and convincing
- Thoroughly **review** your project the day before the examination
  - Make sure your **remember** your design decisions and implementation details.
  - Try to answer **why** you made a design decision